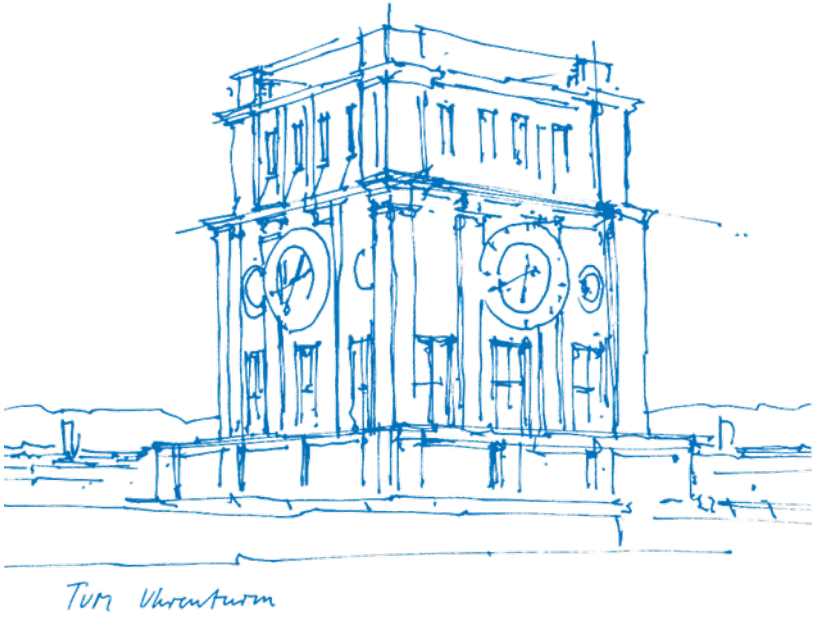
Repetitorium - WS25/26

Patrick Bokelmann, Maxi Heller, Justin Lanfermann, Alisa Parashchenko, Jaden Rotter, Nathan Schmidt, Leon Stark, Bella Tefera

Grundlagen: Betriebssysteme und Systemsoftware (IN0009/IN0034)

Lehrstuhl für Connected Mobility
School of Computation, Information and Technology
Technische Universität München

Garching, 31.01.2026



Disclaimer:

Bei allen Folien handelt es sich um privat erstellte und verwaltete Folien für GBS im WS25/26.

Sie erheben keinerlei Anspruch auf Vollständigkeit und Korrektheit.

Im Zweifel haben IMMER die offiziellen Vorlesungsmaterialien recht!

© F. Sauter & C. Menges & S. Kappes & J. Schönberger & D. Rentz & A. Stephan & P. Bokelmann & T. Jungmann & N. Schmidt & J. Rotter & A. Scheitler & N. Haunschild & M. Heller & J. Lanfermann & A. Parashchenko & L. Stark & B. Tefera

Virtualisierung

Motivation

Ziel: Schaffen einer künstlichen Umgebung

Motivation

Ziel: Schaffen einer künstlichen Umgebung

Retake WiSe 2023/24: *Welche Vorteile bietet Virtualisierung?*

- ISA-Unabhängigkeit
- Höhere Sicherheit (Isolation)
- OS-Unabhängigkeit

Motivation

Ziel: Schaffen einer künstlichen Umgebung

Retake WiSe 2023/24: *Welche Vorteile bietet Virtualisierung?*

- ISA-Unabhängigkeit ✓
- Höhere Sicherheit (Isolation)
- OS-Unabhängigkeit

Motivation

Ziel: Schaffen einer künstlichen Umgebung

Retake WiSe 2023/24: *Welche Vorteile bietet Virtualisierung?*

- ISA-Unabhängigkeit ✓
- Höhere Sicherheit (Isolation) ✓
- OS-Unabhängigkeit

Motivation

Ziel: Schaffen einer künstlichen Umgebung

Retake WiSe 2023/24: *Welche Vorteile bietet Virtualisierung?*

- ISA-Unabhängigkeit ✓
- Höhere Sicherheit (Isolation) ✓
- OS-Unabhängigkeit ✓

Virtuelle Maschinen (1)

Zwei Arten von VMs:

- **System Virtual Machines**

- **Process Virtual Machines**

Virtuelle Maschinen (1)

Zwei Arten von VMs:

- **System Virtual Machines**
  - Umgebung, in der komplettes Gast-Betriebssystem läuft
  - direkt auf Hardware (Typ 1) oder über Host-OS (Typ 2)
  - z.B. QEMU, VirtualBox, VMware Workstation, WSL2, ...
- **Process Virtual Machines**

Virtuelle Maschinen (1)

Zwei Arten von VMs:

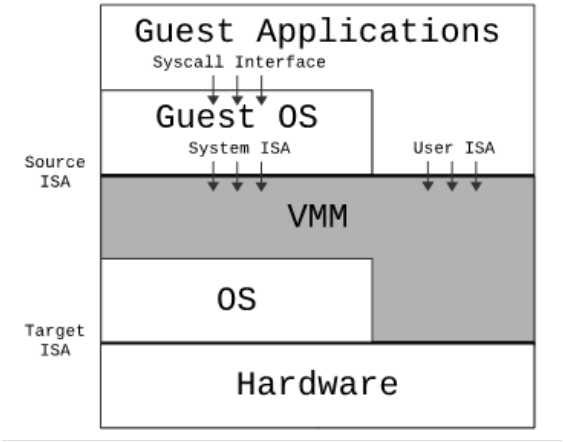
- **System Virtual Machines**
  - Umgebung, in der komplettes Gast-Betriebssystem läuft
  - direkt auf Hardware (Typ 1) oder über Host-OS (Typ 2)
  - z.B. QEMU, VirtualBox, VMware Workstation, WSL2, ...
- **Process Virtual Machines**
  - Umgebung, in der ISA/ABI virtualisiert ist
  - kein zusätzlicher Betriebssystem-Kernel
  - z.B. QEMU, Wine, WSL1, JVM, LLVM, CPython, Valgrind, ...

Virtuelle Maschinen (2)

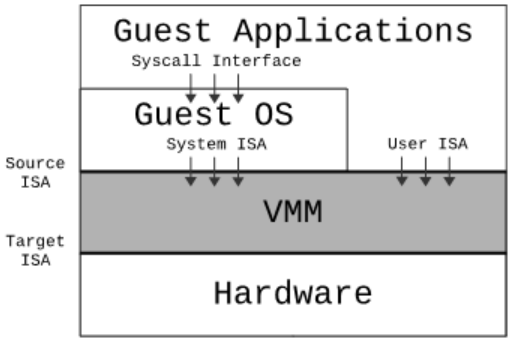
Weise jeder VM-Art den passenden Aufbau zu.

- System VM – Typ 1
- System VM – Typ 2
- Process VM

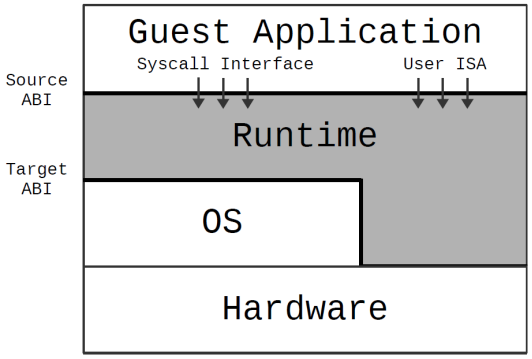
1.



2.



3.

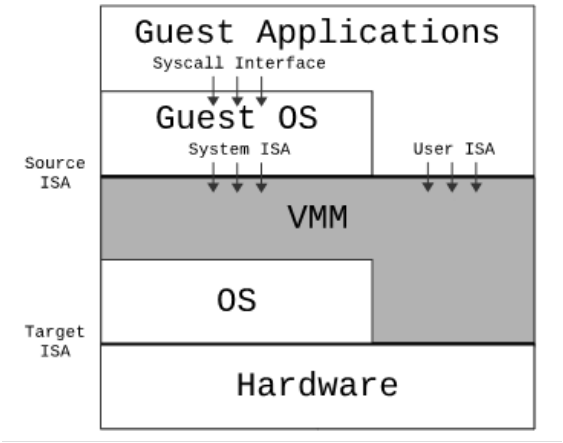


Virtuelle Maschinen (2)

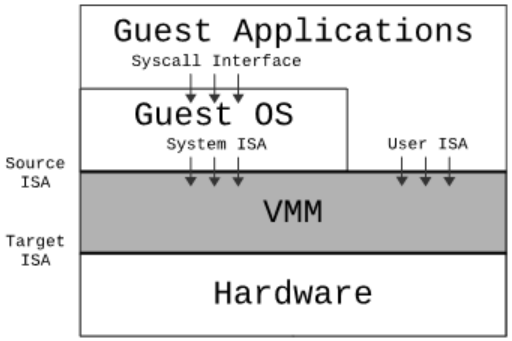
Weise jeder VM-Art den passenden Aufbau zu.

- System VM – Typ 1 $\rightarrow$ 2
- System VM – Typ 2
- Process VM

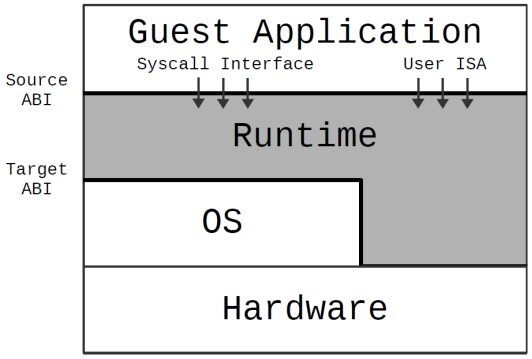
1.



2.



3.

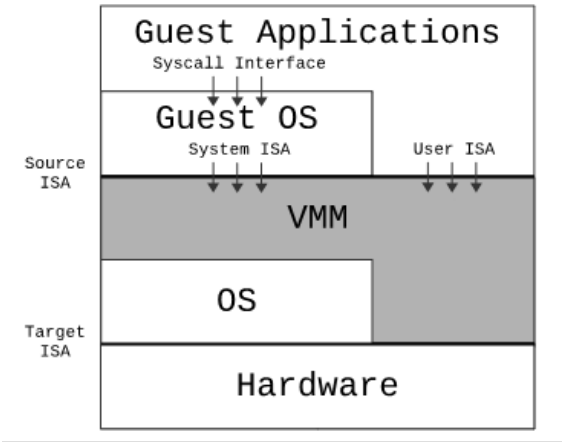


Virtuelle Maschinen (2)

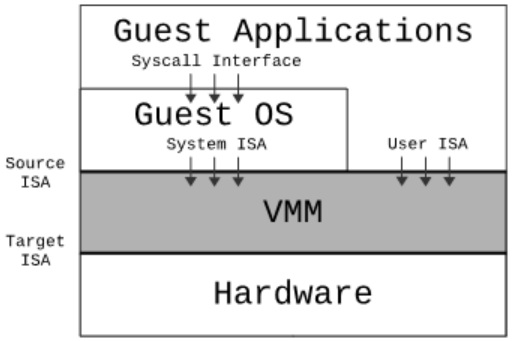
Weise jeder VM-Art den passenden Aufbau zu.

- System VM – Typ 1 $\rightarrow$ 2
- System VM – Typ 2 $\rightarrow$ 1
- Process VM

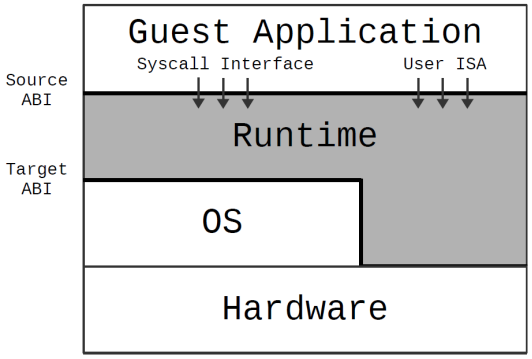
1.



2.



3.

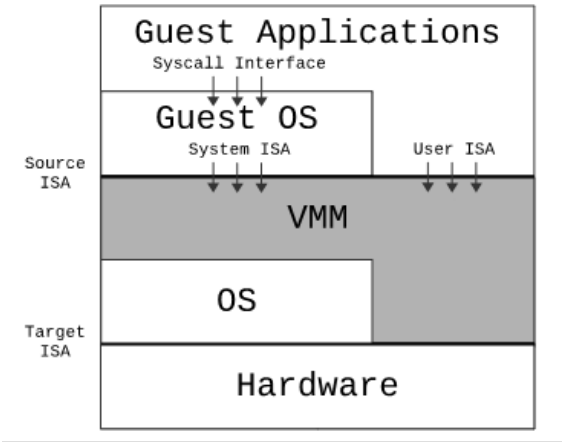


Virtuelle Maschinen (2)

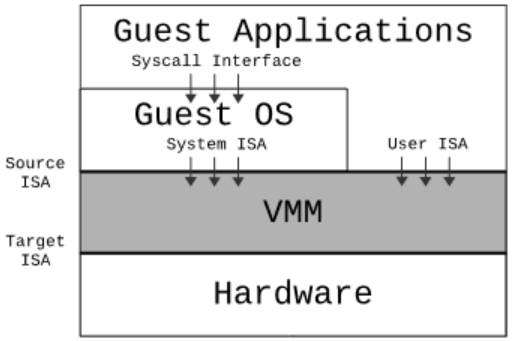
Weise jeder VM-Art den passenden Aufbau zu.

- System VM – Typ 1 → 2
- System VM – Typ 2 → 1
- Process VM → 3

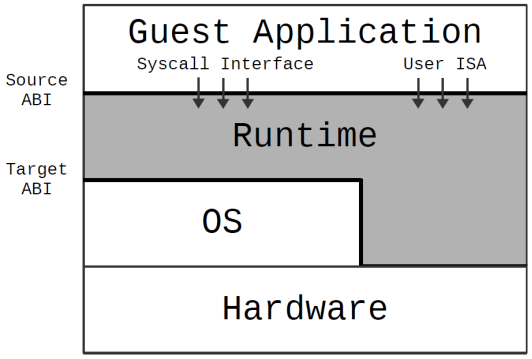
1.



2.



3.



Container – Motivation

Ziele u.A.:

- **Leichtgewichtige Virtualisierung**
  - Kein zweites Gast-Betriebssystem
  - Keine aufwändige ABI/ISA Translation

Container – Motivation

Ziele u.A.:

- **Leichtgewichtige Virtualisierung**
  - Kein zweites Gast-Betriebssystem
  - Keine aufwändige ABI/ISA Translation
- **Reproduzierbare Umgebung**
  - z.B. Softwaretests, Dependencies, einfaches Deployment

Container – Motivation

Ziele u.A.:

- **Leichtgewichtige Virtualisierung**
  - Kein zweites Gast-Betriebssystem
  - Keine aufwändige ABI/ISA Translation
- **Reproduzierbare Umgebung**
  - z.B. Softwaretests, Dependencies, einfaches Deployment
- **Isolierung von Ressourcen**
  - z.B. wegen diversen Sicherheitsaspekten

Linux Namespaces

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?
$\rightarrow$ Zugriffsrechte auf Dateien, bei diversen Syscalls, ...

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?
$\rightarrow$ Zugriffsrechte auf Dateien, bei diversen Syscalls, ...

Aber was ist eigentlich ein User unter Linux?

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?
→ Zugriffsrechte auf Dateien, bei diversen Syscalls, . . .

Aber was ist eigentlich ein User unter Linux?

- Kernel implementiert lediglich die Zugriffsrechte anhand der UIDs
  - Kernel hat ansonsten keine Idee von Nutzern am System (außer 0 = root)

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?
→ Zugriffsrechte auf Dateien, bei diversen Syscalls, . . .

Aber was ist eigentlich ein User unter Linux?

- Kernel implementiert lediglich die Zugriffsrechte anhand der UIDs
  - Kernel hat ansonsten keine Idee von Nutzern am System (außer 0 = root)
- User im klassischen Sinne sind rein im User Space verwaltet
  - Benutzernamen (/etc/passwd), Passwort-Hashes (/etc/shadow), Authentifizierung

User Namespaces – Users und UIDs

Wozu werden Users in Betriebssystemen gebraucht?
→ Zugriffsrechte auf Dateien, bei diversen Syscalls, . . .

Aber was ist eigentlich ein User unter Linux?

- Kernel implementiert lediglich die Zugriffsrechte anhand der UIDs
  - Kernel hat ansonsten keine Idee von Nutzern am System (außer 0 = root)
- User im klassischen Sinne sind rein im User Space verwaltet
  - Benutzernamen (/etc/passwd), Passwort-Hashes (/etc/shadow), Authentifizierung
- Wie funktioniert das?
  - Erinnerung: Jeder Prozess hat eine UID
  - Zugriff-Check (bspw. open()) wird anhand von UID des Prozesses gemacht
  - UID eines Prozesses kann mit setuid() geändert werden, erfordert aber root-Rechte

User Namespaces – Isolation von UIDs

Im initialen User-Namespace existieren zunächst **alle** UIDs

$\rightarrow$ Wie isolieren wir dann UIDs?

User Namespaces – Isolation von UIDs

Im initialen User-Namespace existieren zunächst **alle** UIDs

→ Wie isolieren wir dann UIDs?

Idee: Erzeuge eine Hierarchie

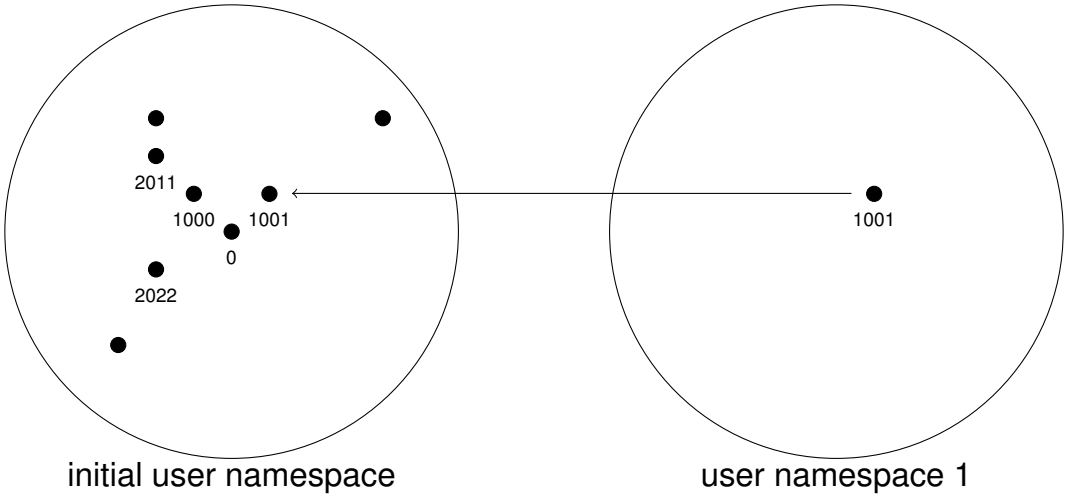
- Im neuen User-Namespace gibt es nur eine Teilmenge von UIDs
- Zwischen User-Namespaces wird dann anhand von einem Mapping übersetzt.
- „Nach außen“: Eindeutig
  - Bspw. wenn wir mit **open()** eine Datei öffnen wollen
- „Nach innen“: Nicht eindeutig → Fallback-UID 65534 (nobody)
  - Bspw. wenn wir mit **stat()** UID-Informationen aus der Inode einer Datei auslesen

User Namespaces Beispiel

```
name  uid mode
A.txt 0 rw- --- ---
B.txt 1000 rw- --- ---
C.txt 1001 rw- --- ---
```

0: root
1000: alice
1001: bob
2011: userA
2022: userB
65534: nobody

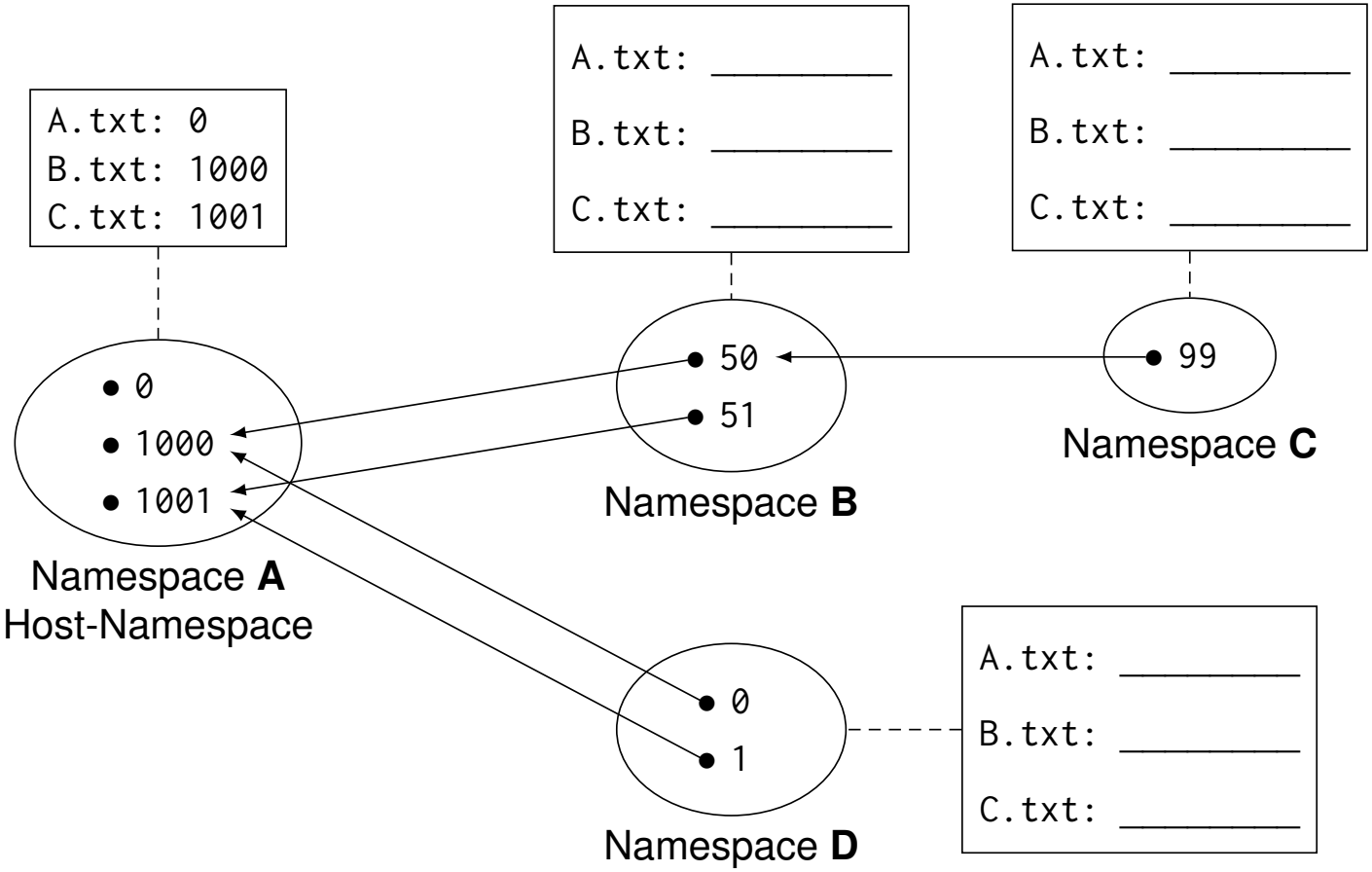
Tabelle: /etc/passwd



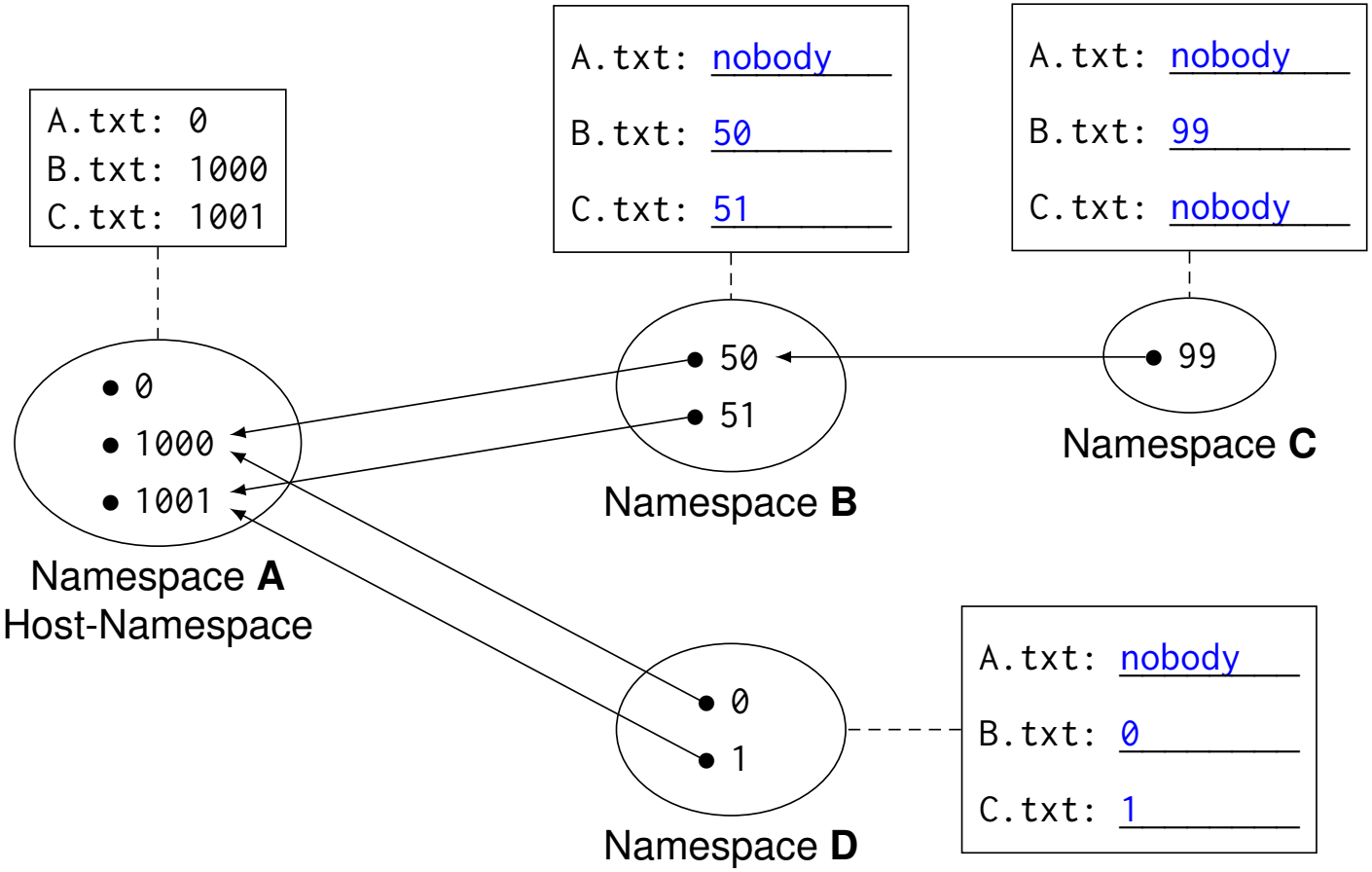
User Namespaces

Demo

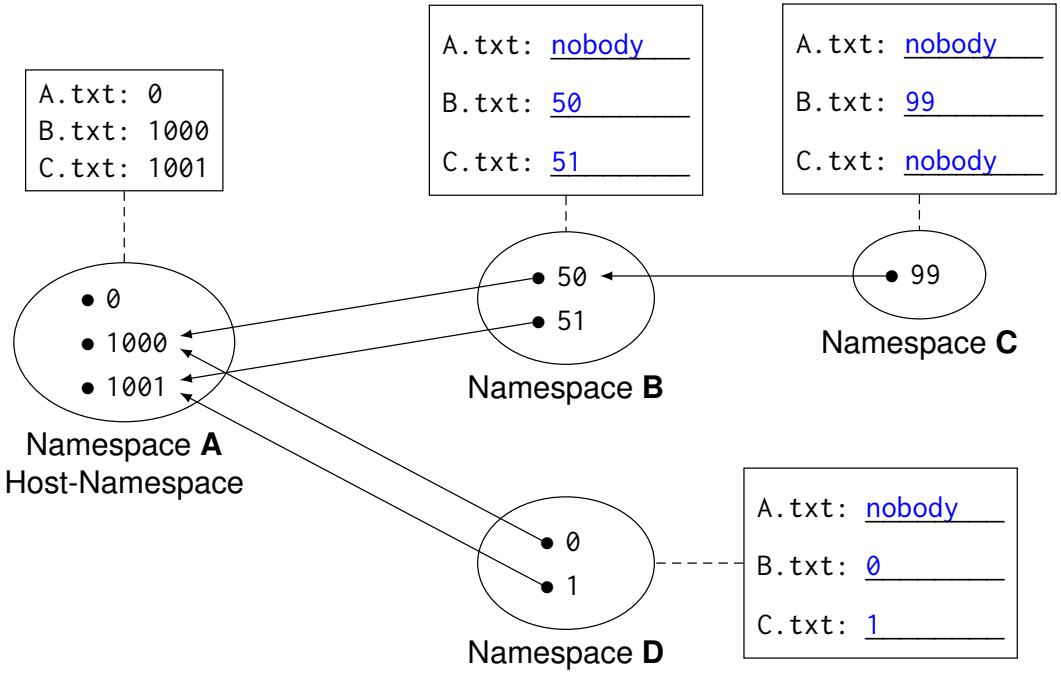
Retake 2023 Aufgabe 9



Retake 2023 Aufgabe 9

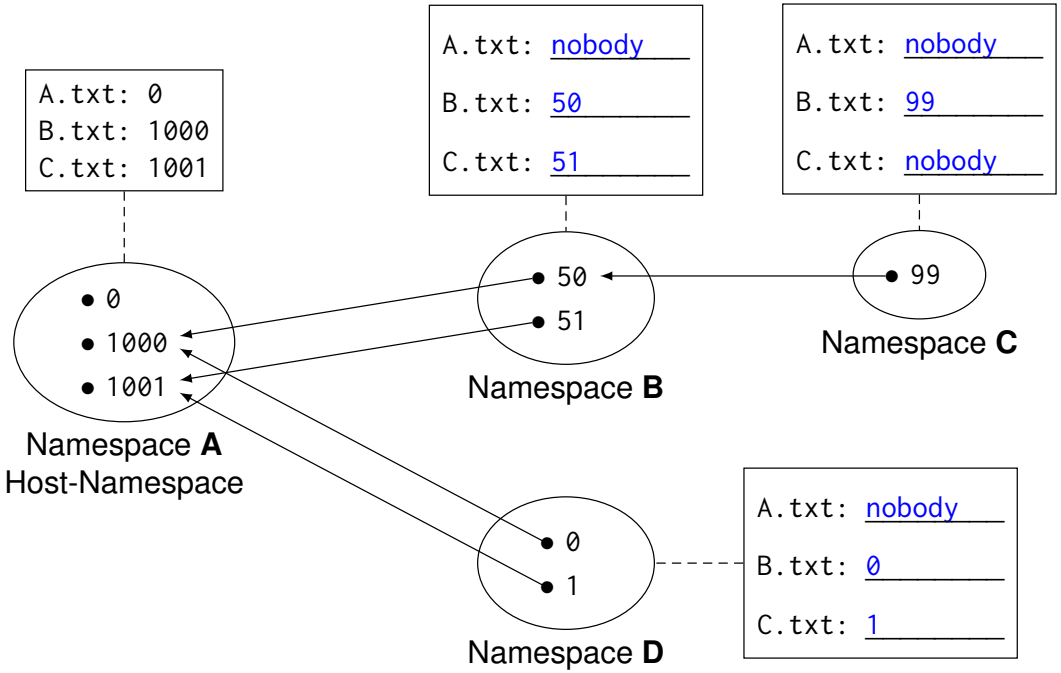


Retake 2023 Aufgabe 9



Angenommen, Nutzer 51 legt im Namespace B eine Datei an. Diese habe die Rechte 6008 = rw——-, d.h. nur der Besitzer darf auf die Datei zugreifen. Darf Nutzer 1 im Namespace D diese Datei lesen?

Retake 2023 Aufgabe 9



Angenommen, Nutzer 51 legt im Namespace B eine Datei an. Diese habe die Rechte 6008 = rw——-, d.h. nur der Besitzer darf auf die Datei zugreifen. Darf Nutzer 1 im Namespace D diese Datei lesen?

Außen (im Namespace A) gehört diese Datei dem Nutzer 1001, diese Datei erscheint also im Namespace D mit dem Eigentümer 1. Damit darf Nutzer 1 im Namespace D davon lesen.

Mount Namespaces – Isolation von mount points

- Mount Namespaces isolieren die Liste der **mount points**

```
cat proc/self/mounts
```

- Keine Hierarchie wie bei User Namespaces
- Bei Erstellung eines Mount Namespaces wird eine Kopie der Liste der mount points erstellt
- Alle weiteren Änderungen an der Liste der Mount Points sind nur im Mount Namespace sichtbar

PID Namespaces – Isolation von Process IDs

- PID Namespaces isolieren die Liste der **Process IDs**
- PID Namespaces haben eine Hierarchie
- Erster Prozess in PID Namespace bekommt PID 1

PID Namespaces – Isolation von Process IDs

- PID Namespaces isolieren die Liste der **Process IDs**
- PID Namespaces haben eine Hierarchie
- Erster Prozess in PID Namespace bekommt PID 1
- Kleine Eigenheit: PID eines Prozesses kann sich nicht ändern
- Daher bekommt erst der erste Kindprozess nach dem erzeugen des PID Namespaces die PID 1

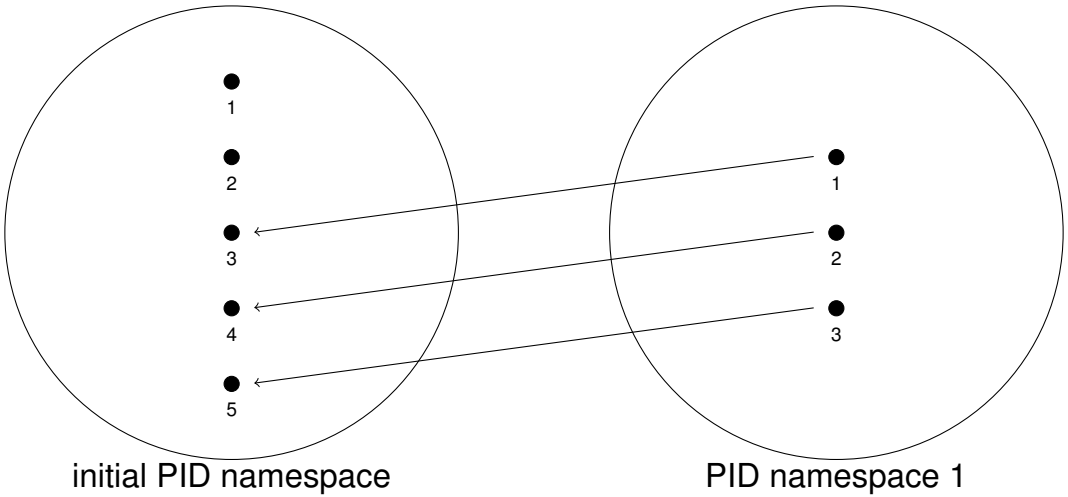
PID Namespaces – Isolation von Process IDs

- PID Namespaces isolieren die Liste der **Process IDs**
- PID Namespaces haben eine Hierarchie
- Erster Prozess in PID Namespace bekommt PID 1
- Kleine Eigenheit: PID eines Prozesses kann sich nicht ändern
- Daher bekommt erst der erste Kindprozess nach dem erzeugen des PID Namespaces die PID 1
- Beispiele: Wer kann welche PID sehen?

PID Namespaces – Isolation von Process IDs

- PID Namespaces isolieren die Liste der **Process IDs**
- PID Namespaces haben eine Hierarchie
- Erster Prozess in PID Namespace bekommt PID 1
- Kleine Eigenheit: PID eines Prozesses kann sich nicht ändern
- Daher bekommt erst der erste Kindprozess nach dem erzeugen des PID Namespaces die PID 1
- Beispiele: Wer kann welche PID sehen?
- . . . und was ist mit `htop`?

PID Namespaces Beispiel



PID Namespaces

Demo